Manual de Arquitetura

IgnisGeo — Sistema Web de Priorização Geoespacial de Áreas de Risco de Queimadas

V1.2026

Backend	Django 5 + GeoDjango + DRF
Banco	PostgreSQL 16 + PostGIS 3.4
Async	Celery 5 + Redis 7
Frontend	Vue 3 + Pinia + Leaflet + Canvas 2D
Infra	Docker Compose (5 containers)
Testes	46 testes automatizados · 0 falhas
Docs	pdoc · docstrings estilo NumPy
Dados	3.876.956 focos · 27 UF · 16 meses

Rogério Cerqueira
Engenharia de Computação — UEFS · TCC 2026.1

Sumário

1 Visão Geral da Arquitetura **2**
1.1 Diagrama de Arquitetura em Camadas . . . 2
1.2 Responsabilidades por Camada . . . 2

2 Stack Tecnológico Completo **2**
2.1 Backend . . . 2
2.2 Banco de Dados . . . 2
2.3 Frontend . . . 2
2.4 Infraestrutura . . . 2

3 Docker e Infraestrutura **2**
3.1 Docker Compose — Estrutura . . . 2
3.2 Diagrama de Containers (C4 Model) . . . 5
3.3 Decisões de Infraestrutura . . . 6

4 PostgreSQL 16 e PostGIS 3.4 **6**
4.1 Models Geoespaciais . . . 6
4.2 Queries PostGIS Utilizadas . . . 7
4.3 Bulk Create — Estratégia de Inserção em Lote . . . 7

5 Celery 5 + Redis 7 **7**
5.1 Arquitetura Produtor-Consumidor . . . 8
5.2 Task de Importação . . . 8
5.3 Por que o TOPSIS não usa Celery . . . 9

6 API REST — Django REST Framework **9**
6.1 Endpoints . . . 9
6.2 Paginação . . . 9
6.3 Decisão de Baixo Acoplamento Externo . . . 10

7 Diagramas de Sequência **10**
7.1 Sequência 1 — Pipeline TOPSIS Fuzzy . . . 10
7.2 Sequência 2 — Importação Assíncrona CSV . . . 11
7.3 Sequência 3 — Tela de Resultados e Gráficos . . . 11

8 Frontend — Vue 3, Pinia e Canvas 2D **11**
8.1 Gerenciamento de Estado com Pinia . . . 11
8.2 Gráficos Canvas 2D — G1 a G8 . . . 12

9 Documentação — Docstrings Estilo NumPy e pdoc **12**
9.1 Padrão NumPy de Docstring . . . 12
9.2 Geração da Documentação com pdoc . . . 14
9.3 Estrutura da Documentação Gerada . . . 14

10 Testes Automatizados — Robustez e Cobertura **14**
10.1 Suite Completa: 46 Testes, 0 Falhas, 0,330s . . . 14

10.2 Por que SimpleTestCase na Suite de Sensibilidade 14
10.3 Resultado da Análise de Sensibilidade OAT 14
10.4 Comando de Execução 14

11 Robustez e Decisões de Qualidade 16
11.1 Mapa de Decisões Arquiteturais 16
11.2 Proteção contra Estado Parcial 16
11.3 Robustez dos Endpoints GET 17

12 Estrutura de Diretórios do Projeto 17

13 Resumo da Arquitetura Final 18

1. Visão Geral da Arquitetura

O IgnisGeo adota uma **arquitetura em camadas** (*Layered Architecture*) combinada com o padrão **orientado a eventos** (*Event-Driven*) para o processamento assíncrono de dados. A separação entre camadas garante que cada componente tenha uma única responsabilidade (SRP – *Single Responsibility Principle*).

1.1. Diagrama de Arquitetura em Camadas

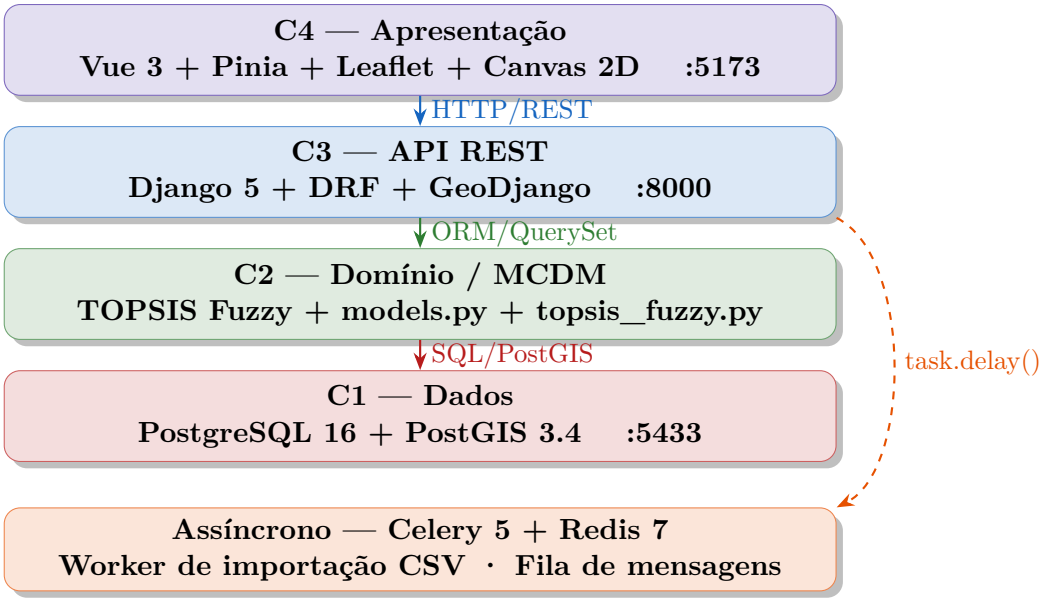

Figura 1: Arquitetura em 4 camadas do IgnisGeo com canal assíncrono Celery.

1.2. Responsabilidades por Camada

2. Stack Tecnológico Completo

2.1. Backend

2.2. Banco de Dados

2.3. Frontend

2.4. Infraestrutura

3. Docker e Infraestrutura

3.1. Docker Compose — Estrutura

Listing 1: Estrutura do docker-compose.yml do IgnisGeo.

```
version: "3.9"
services:
```

Tabela 1: Camadas, padrões e responsabilidades do IgnisGeo.

Camada	**Tecnologia**	**Padrão**	**Responsabilidade**
C4 Apresentação	Vue 3 + Pinia	MVVM · Observer	Renderização, estado global, interação
C3 API REST	Django 5 + DRF	Facade · Adapter	Exposição de endpoints, serialização, paginação
C2 Domínio	Python puro	Strategy · SRP	TOPSIS Fuzzy, correlação, regras de negócio
C1 Dados	PostgreSQL + PostGIS	Repository	Persistência, queries geoespaciais, índices
Async	Celery + Redis	Command · Queue	Importação CSV, tarefas em segundo plano

Tabela 2: Tecnologias do backend e suas funções no IgnisGeo.

Tecnologia	**Versão**	**Função**
Python	3.12	Linguagem principal do backend
Django	5.x	Framework web, ORM, admin, migrations
GeoDjango	nativo	Extensão geoespacial do Django (PostGIS)
DRF	3.x	Serializers, ViewSets, paginação, APIClient
Psycopg2	2.x	Driver Python-PostgreSQL
Celery	5.x	Fila de tarefas assíncronas
Redis	7.x	Message broker do Celery
pdoc	16.x	Geração automática de documentação HTML

Tabela 3: Tecnologias de persistência do IgnisGeo.

Tecnologia	**Versão**	**Função**
PostgreSQL	16	SGBD relacional principal
PostGIS	3.4	Extensão geoespacial: tipos Point, geometry, índice GIST
GEOS	3.x	Biblioteca de geometrias (dep. do PostGIS)
GDAL	3.x	Abstração de dados geoespaciais (dep. do GeoDjango)

Tabela 4: Tecnologias do frontend e suas funções.

Tecnologia	**Versão**	**Função**
Vue 3	3.x	Framework reativo (Composition API)
Pinia	2.x	Gerenciamento de estado global
Leaflet	1.x	Mapa interativo com camadas GeoJSON e heatmap
Canvas 2D	nativo	Gráficos G1–G8 sem dependência de biblioteca
Axios	1.x	Cliente HTTP para consumo da API REST
Vite	5.x	Bundler e servidor de desenvolvimento
leaflet.heat	0.2.x	Plugin de heatmap para Leaflet

Tabela 5: Infraestrutura de containers do IgnisGeo.

Container	**Imagem**	**Porta**	**Função**
backend	python:3.12-slim	8000	API Django + DRF
frontend	node:20-alpine	5173	Vue 3 + Vite dev server
db	postgis/postgis:16-3.4	5433	PostgreSQL + PostGIS
redis	redis:7-alpine	6379	Broker Celery
worker	python:3.12-slim	—	Celery Worker (importação CSV)

```yaml
db:
  image: postgis/postgis:16-3.4
  environment:
    POSTGRES_DB: queimadas
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: ${DB_PASSWORD}
  ports: ["5433:5432"]
  volumes: [pgdata:/var/lib/postgresql/data]

redis:
  image: redis:7-alpine
  ports: ["6379:6379"]

backend:
  build: ./backend
  command: python manage.py runserver 0.0.0.0:8000
  ports: ["8000:8000"]
  depends_on: [db, redis]
  environment:
    DATABASE_URL: postgis://postgres:${DB_PASSWORD}@db:5432/
       queimadas
    CELERY_BROKER_URL: redis://redis:6379/0

worker:
  build: ./backend
  command: celery -A backend worker -l info
  depends_on: [db, redis]

frontend:
  build: ./frontend
  command: npm run dev -- --host
  ports: ["5173:5173"]
  depends_on: [backend]
```

3.2. Diagrama de Containers (C4 Model)

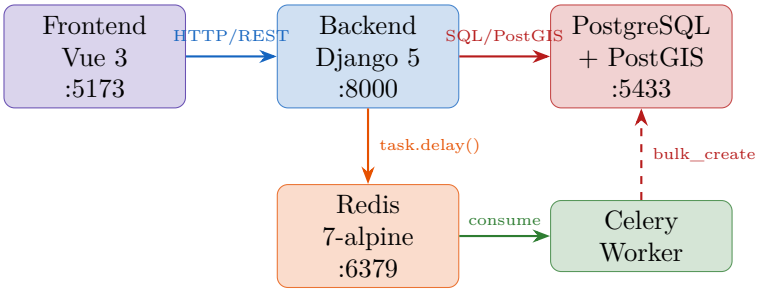

Figura 2: Diagrama de containers C4 do IgnisGeo. Setas sólidas = chamadas síncronas; tracejadas = assíncronas.

3.3. Decisões de Infraestrutura

Por que postgis/postgis:16-3.4 e não postgres:16?

O PostGIS não é instalável em runtime sem permissão de superusuário no container. A imagem oficial `postgis/postgis` já inclui a extensão compilada e a habilita automaticamente via `CREATE EXTENSION IF NOT EXISTS postgis`, executado pelo GeoDjango no `prepare_database()`.

Por que Redis e não RabbitMQ como broker?

Redis tem imagem Alpine com 40 MB contra 200+ MB do RabbitMQ. Para o volume atual (importações periódicas, não contínuas), Redis como broker é suficiente e reduz a complexidade de configuração. A persistência do resultado das tasks (*Result Backend*) é opcional e não foi habilitada — o sistema retorna o `task_id` ao cliente e o polling é responsabilidade da aplicação.

4. PostgreSQL 16 e PostGIS 3.4

4.1. Models Geoespaciais

Listing 2: Models principais do IgnisGeo com campos geoespaciais.

```
from django.contrib.gis.db import models

class FocoQueimada(models.Model):
    localizacao = models.PointField(srid=4326)  # WGS-84
    data_hora   = models.DateTimeField()
    municipio   = models.CharField(max_length=100)
    estado      = models.CharField(max_length=2)
    bioma       = models.CharField(max_length=50)
    frp                 = models.FloatField(null=True)
    risco_historico     = models.FloatField(null=True)
    dias_sem_chuva      = models.FloatField(null=True)
    precipitacao        = models.FloatField(null=True)
    satelite            = models.CharField(max_length=50, null=True)

    @property
    def latitude(self):  return self.localizacao.y
    @property
    def longitude(self): return self.localizacao.x

class AreaRisco(models.Model):
    nome        = models.CharField(max_length=200)
    estado      = models.CharField(max_length=2)
    bioma       = models.CharField(max_length=50)
    geometria   = models.GeometryField(srid=4326, null=True)
    # Centroid(Union(FocoQueimada.localizacao)) por municipio
    score_topsis          = models.FloatField(decimal_places=4)
    ranking               = models.IntegerField()
    nivel_risco           = models.CharField(max_length=10)
    total_focos           = models.IntegerField()
    frp_media             = models.FloatField(null=True)
    risco_historico_medio = models.FloatField(null=True)
```

```
    dias_sem_chuva_medio   = models.FloatField(null=True)
    precipitacao_media     = models.FloatField(null=True)
    periodo_inicio         = models.DateField()
    periodo_fim            = models.DateField()
    atualizado_em          = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["ranking"]
```

4.2. Queries PostGIS Utilizadas

Tabela 6: Funções PostGIS utilizadas no IgnisGeo e suas finalidades.

Função PostGIS	Uso no IgnisGeo
`Centroid(Union(localizacao))`	Calcula o centroide geográfico dos focos por município, eliminando necessidade de shapefile IBGE
`ST_Within(localizacao, bbox)`	Filtro de focos dentro de um *bounding box* fornecido pelo frontend
`CREATE EXTENSION postgis`	Habilitado automaticamente pelo GeoDjango no `prepare_database()`
`GIST index`	Índice espacial criado automaticamente pelo GeoDjango para o campo `PointField`

4.3. Bulk Create — Estratégia de Inserção em Lote

Definição 4.1 — Bulk Create

Em vez de N transações individuais, o `bulk_create` emite um único `INSERT` com todos os registros:

Estratégia	Queries	Tempo estimado
`create()` individual	3.876.956	3–4 horas
`bulk_create(batch=5000)`	$\approx$ 776	3–8 minutos

Listing 3: Padrão bulk_create na importação de focos.

```
# Importacao: ignore_conflicts=True para reimportacoes seguras
FocoQueimada.objects.bulk_create(
    lote, batch_size=5000, ignore_conflicts=True
)

# Ranking TOPSIS: delete-then-insert atomico
AreaRisco.objects.all().delete()
AreaRisco.objects.bulk_create(novas_areas)
# A API nunca expoem estado parcial entre as duas operacoes
```

5. Celery 5 + Redis 7

5.1. Arquitetura Produtor-Consumidor

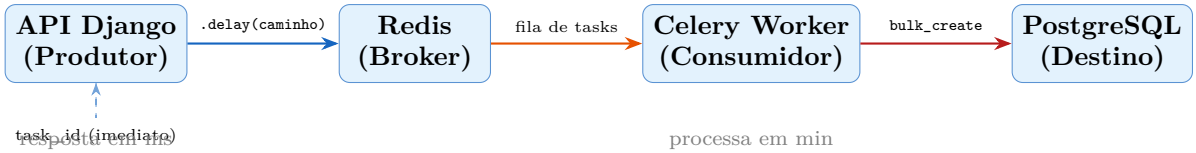

Figura 3: Fluxo assíncrono de importação CSV via Celery.

5.2. Task de Importação

Listing 4: Task Celery para importação de CSV do INPE.

```
from celery import shared_task

@shared_task
def importar_csv_inpe(caminho: str) -> dict:
    """
    Importa focos de queimada de um CSV do BDQueimadas/INPE.

    Parameters
    ----------
    caminho : str
        Caminho absoluto para o arquivo CSV exportado do INPE.

    Returns
    -------
    dict
        Dicionario com chaves 'importados', 'ignorados', 'erros'.

    Notes
    -----
    Usa bulk_create com batch_size=5000 e ignore_conflicts=True
    para suportar reimportacoes parciais sem duplicatas.
    """
    import csv
    from django.contrib.gis.geos import Point
    lote, importados, erros = [], 0, 0
    with open(caminho, encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                lote.append(FocoQueimada(
                    localizacao=Point(
                        float(row["longitude"]),
                        float(row["latitude"]), srid=4326),
                    ...
                ))
                if len(lote) >= 5000:
                    FocoQueimada.objects.bulk_create(
                        lote, ignore_conflicts=True)
                    importados += len(lote); lote = []
            except Exception:
                erros += 1
    if lote:
        FocoQueimada.objects.bulk_create(lote,ignore_conflicts=True)
        importados += len(lote)
```

```
    return {"importados": importados, "erros": erros}
```

5.3. Por que o TOPSIS não usa Celery

O cálculo do TOPSIS Fuzzy sobre 6.195 áreas completa em menos de 2 segundos — compatível com o ciclo de requisição-resposta da API. A assincronicidade só é necessária para operações que bloqueiam por mais de 5–10 segundos. Manter o TOPSIS síncrono simplifica o debugging e elimina a necessidade de polling de status.

6. API REST — Django REST Framework

6.1. Endpoints

Tabela 7: Endpoints da API REST do IgnisGeo.

Endpoint	Método	Descrição
`/api/focos/geojson/`	GET	GeoJSON de focos (máx. 5.000) com bbox
`/api/focos/`	GET	Lista de focos sem geometria
`/api/ranking/`	GET	Ranking paginado (10/página, máx. 100)
`/api/calcular-topsis/`	POST	Executa pipeline TOPSIS Fuzzy
`/api/serie-temporal/`	GET	Focos agregados por mês (TruncMonth)
`/api/graficos-dados/`	GET	7 datasets em 1 requisição
`/api/correlacao/`	GET	Pearson + Spearman 5×5 com p-valores
`/api/estatisticas/`	GET	Totais globais e por bioma
`/api/importar-csv/`	POST	Dispara task Celery de importação
`/api/areas-risco/geojson/`	GET	GeoJSON das áreas ranqueadas

6.2. Paginação

Listing 5: Configuração de paginação do endpoint de ranking.

```
class RankingPagination(PageNumberPagination):
    page_size              = 10
    page_size_query_param = "page_size"
    max_page_size          = 100

class RankingView(generics.ListAPIView):
    serializer_class = AreaRiscoRankingSerializer
    pagination_class = RankingPagination

    def get_queryset(self):
        qs = AreaRisco.objects.all()
        # Filtros opcionais: nivel_risco, estado, bioma
        for param, field in [
            ("nivel_risco","nivel_risco"),
```

```
            ("estado","estado"),
            ("bioma","bioma"),
        ]:
            val = self.request.query_params.get(param)
            if val:
                qs = qs.filter(**{field: val.upper()})
        return qs.order_by("ranking")
```

6.3. Decisão de Baixo Acoplamento Externo

Decisão de projeto: o núcleo analítico do IgnisGeo — algoritmo TOPSIS Fuzzy, correlação de Pearson e Spearman, p-valores via fração beta incompleta (método de Lentz) — foi implementado exclusivamente com a biblioteca padrão do Python (`math`, nenhum pacote científico externo). Isso reduz o acoplamento a ecossistemas de terceiros (NumPy, SciPy), simplifica o ambiente de execução e elimina riscos de incompatibilidade por atualização de dependências. Django e DRF atuam estritamente como camada de exposição — sem participação na lógica de decisão multicritério.

7. Diagramas de Sequência

7.1. Sequência 1 — Pipeline TOPSIS Fuzzy

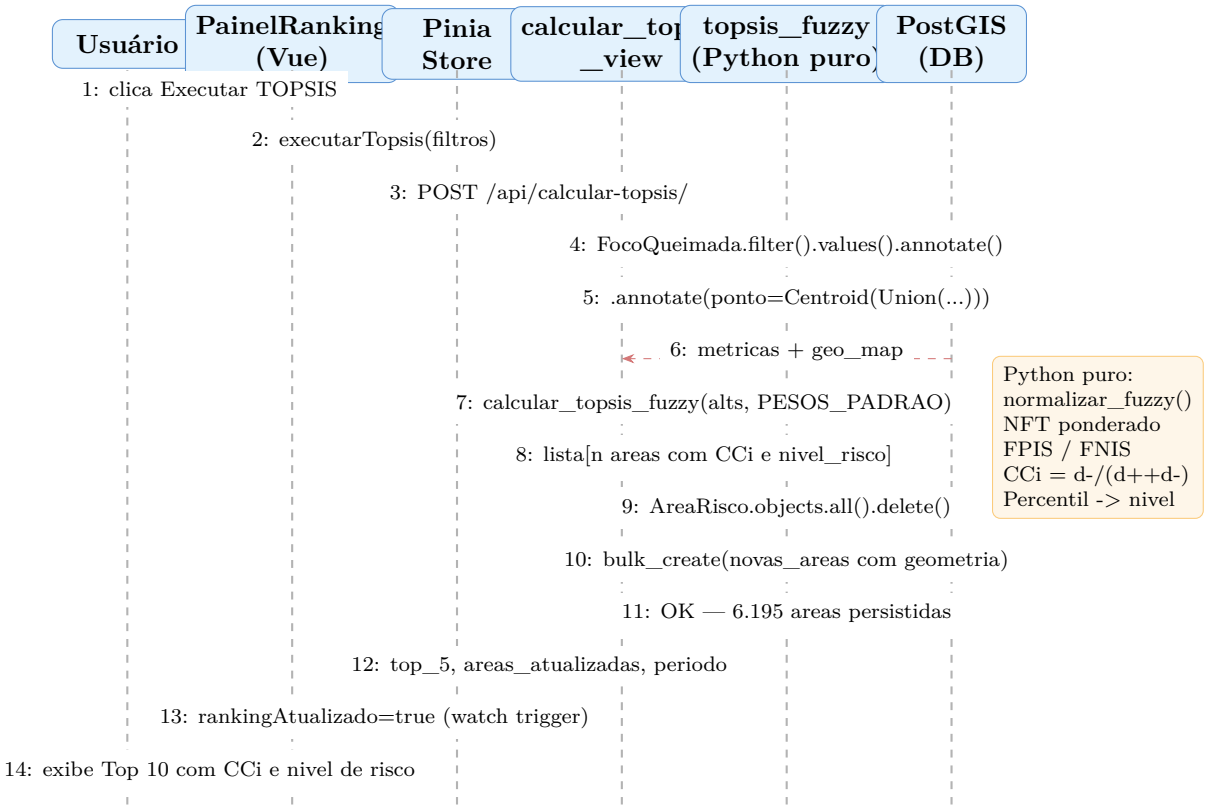

Figura 4: Diagrama de sequência completo do pipeline TOPSIS Fuzzy.

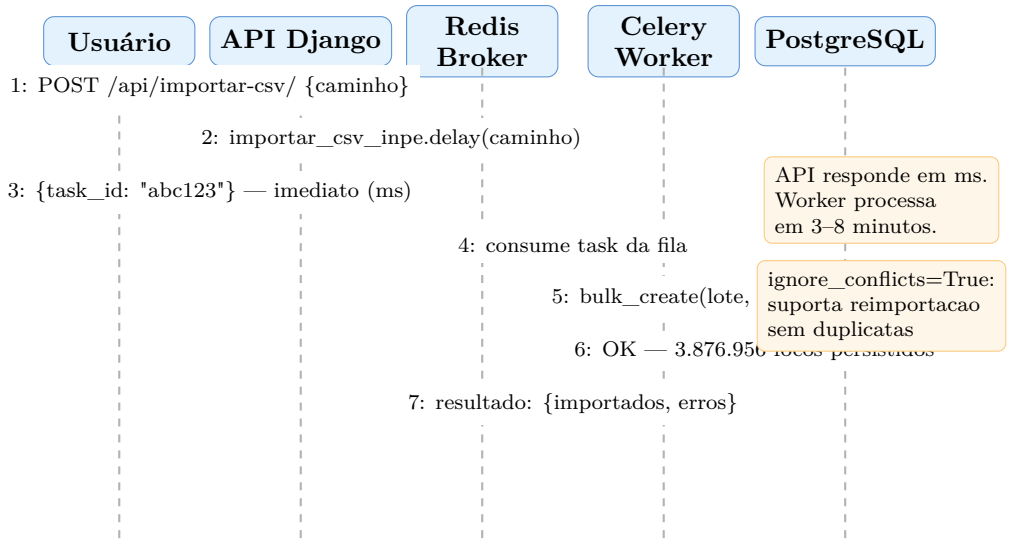

Figura 5: Sequência da importação assíncrona de CSV via Celery.

7.2. Sequência 2 — Importação Assíncrona CSV

7.3. Sequência 3 — Tela de Resultados e Gráficos

8. Frontend — Vue 3, Pinia e Canvas 2D

8.1. Gerenciamento de Estado com Pinia

Listing 6: Store Pinia do IgnisGeo (queimadas.js) — estrutura principal.

```
// stores/queimadas.js
export const useQueimadasStore = defineStore("queimadas", {
  state: () => ({
    focosGeoJSON:       null,
    areasGeoJSON:       null,
    rankingItems:       [],
    rankingPagina:      1,
    rankingTotal:       0,
    rankingAtualizado:  false,
    totalFocos:         0,
    carregando:         false,
    filtros: {
      estado: "", bioma: "",
      nivelRisco: "", dataInicio: "", dataFim: "",
    },
  }),
  actions: {
    async carregarRanking(pagina = 1) {
      const params = this.filtrosAtivos();
      params.page  = pagina;
      const resp   = await api.get("/ranking/", { params });
      this.rankingItems = resp.data.results;
      this.rankingTotal = resp.data.count;
      this.rankingPagina = pagina;
    },
    filtrosAtivos() {
```

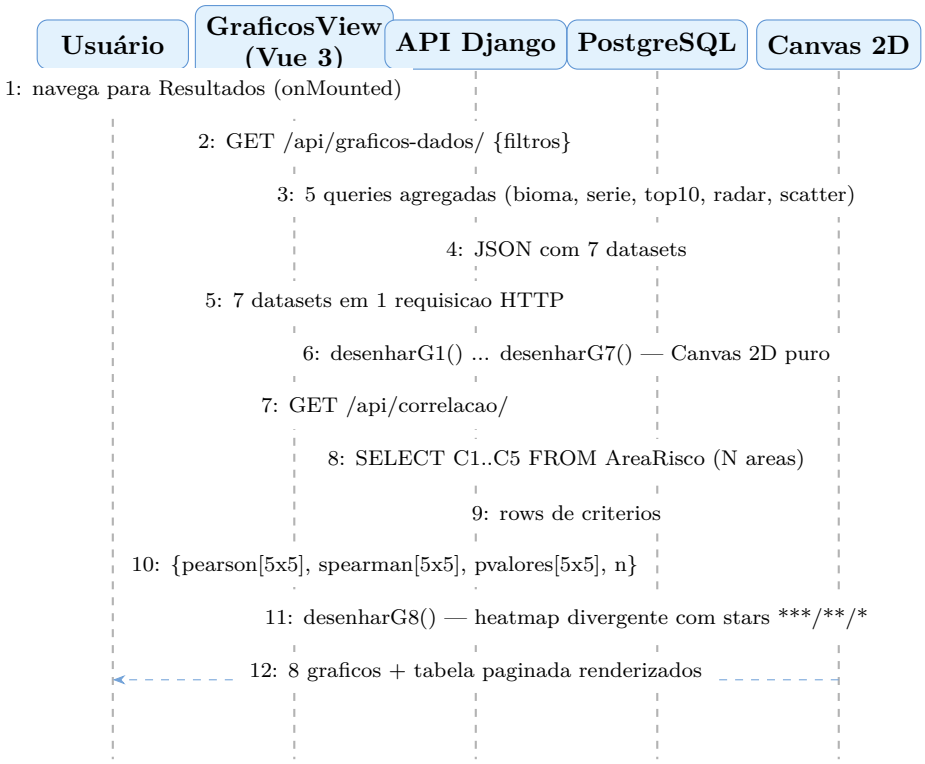

Figura 6: Sequência completa da tela de Resultados com 8 gráficos.

```
      // Conversao camelCase -> snake_case para o DRF
      const map = {
        nivelRisco: "nivel_risco",
        dataInicio: "data_inicio",
        dataFim:    "data_fim",
      };
      const params = {};
      for (const [k, v] of Object.entries(this.filtros)) {
        if (v) params[map[k] || k] = v;
      }
      return params;
    },
  },
});
```

8.2. Gráficos Canvas 2D — G1 a G8

9. Documentação — Docstrings Estilo NumPy e pdoc

9.1. Padrão NumPy de Docstring

O módulo `topsis_fuzzy.py` é completamente documentado no padrão NumPy, que usa seções nomeadas separadas por sublinhados:

Listing 7: Docstring estilo NumPy da função principal do IgnisGeo.

```
def calcular_topsis_fuzzy(alternativas: list, pesos: dict = None) ->
    list:
```

Tabela 8: Gráficos implementados com Canvas 2D nativo no IgnisGeo.

ID	Tipo	Dados e destaques técnicos
G1	Barras horizontais	Focos por bioma + FRP médio; clipping por coluna
G2	Linha + área	Série temporal com eixos rotulados e gridlines
G3	Barras horizontais	Top 10 com colunas fixas e `ctx.clip()`
G4	Radar	Top 5 normalizado $[0, 1]$; polígono polar multicritério
G5	Scatter	Score $\times$ focos em escala log; 4 cores por nível
G6	Histograma + Gauss	Densidade $f_b = n_b/(N \cdot \Delta)$; curva $N(\mu, \sigma)$
G7	Boxplot	Quartis, IQR, whiskers 1,5 $\times$ IQR, outliers
G8	Heatmap	Correlação 5×5; colormap azul-branco-vermelho

```
    """
    Executa o algoritmo TOPSIS Fuzzy (Chen, 2000).

    Parameters
    ----------
    alternativas : list[dict]
        Lista de municipios com os 5 criterios do INPE.
        Cada dicionario deve conter as chaves de CRITERIOS.
    pesos : dict, optional
        Mapeamento {criterio: (l, m, u)} com NFT.
        Se None, usa PESOS_PADRAO do modulo.

    Returns
    -------
    list[dict]
        Lista ordenada por ranking (1=maior risco) com campos:
        nome, score_topsis, ranking, nivel_risco e os 5 criterios.

    Notes
    -----
    Classificacao por percentil: max(1, n*p) garante >= 1 CRITICO
    para qualquer n >= 1. O criterio C5 (precipitacao) e invertido
    semanticamente antes da normalizacao (criterio de custo).

    References
    ----------
    Chen, C.T. (2000). Extensions of the TOPSIS for group
    decision-making under fuzzy environment.
    Fuzzy Sets and Systems, 114(1), 1-9.

    Examples
    --------
    >>> alts = [{"nome": "A", "total_focos": 100,
    ...           "frp_media": 50.0, "risco_historico_medio": 0.5,
    ...           "dias_sem_chuva_medio": 20.0, "precipitacao_media":
        5.0}]
    >>> resultado = calcular_topsis_fuzzy(alts)
    >>> resultado[0]["ranking"]
    1
```

```
    """
```

9.2. Geração da Documentação com pdoc

Listing 8: Comandos para gerar a documentação HTML do IgnisGeo.

```
# Instala o pdoc no ambiente virtual
pip install pdoc

# Documenta apenas o modulo de algoritmo
pdoc queimadas/topsis_fuzzy.py --output-dir backend/docs/

# Documenta todo o app queimadas
pdoc queimadas --output-dir backend/docs/

# Abre no navegador
xdg-open backend/docs/queimadas/topsis_fuzzy.html
```

9.3. Estrutura da Documentação Gerada

```
backend/docs/
    index.html
    queimadas/
        topsis_fuzzy.html   <- NumeroFuzzy, normalizar_fuzzy,
                               PESOS_LINGUISTICOS, calcular_topsis_fuzzy
search.js
```

O pdoc detecta automaticamente o estilo NumPy e renderiza as seções `Parameters`, `Returns`, `Notes`, `References` e `Examples` com formatação HTML navegável.

10. Testes Automatizados — Robustez e Cobertura

10.1. Suite Completa: 46 Testes, 0 Falhas, 0,330s

10.2. Por que SimpleTestCase na Suite de Sensibilidade

Os 5 testes de análise de sensibilidade chamam exclusivamente `calcular_topsis_fuzzy()` — Python puro sem acesso ao banco. Usar `SimpleTestCase` em vez de `TestCase`:

- Elimina a criação/destruição do banco de teste por classe
- Reduz o tempo de execução dos 5 testes de ∼1s para ∼5ms
- Executa sem Docker ativo (sem PostGIS disponível na máquina local)

10.3. Resultado da Análise de Sensibilidade OAT

10.4. Comando de Execução

Listing 9: Comandos para executar a suite de testes do IgnisGeo.

```
# Suite completa (requer Docker com PostGIS)
docker compose exec backend python manage.py test queimadas -v 2

# Apenas sensibilidade (sem Docker, sem banco)
```

Tabela 9: Classes de teste, cobertura e tipo de TestCase.

Classe	**Cobre**	**Base**	**Tests**
NumeroFuzzyTest	NFT: dist. vértice, simetria, fórmula exata	TestCase	5
NormalizarFuzzyTest	Limites $[0, 1]$, degenerado, spread ± 0.10	TestCase	5
ClassificarNivelTest	Limiares fixos de CC_i	TestCase	4
TopsisTest	Pipeline completo, percentil, rank sequencial	TestCase	9
FocoQueimadaModelTest	Model + PostGIS + properties	TestCase	4
AreaRiscoModelTest	Model + ordenação, str()	TestCase	3
EstatisticasAPITest	GET /estatisticas/ com/sem dados	TestCase	2
FocosAPITest	Filtros, GeoJSON, FeatureCollection	TestCase	4
RankingAPITest	Paginação, filtros, ordering	TestCase	2
TopsisAPITest	POST com focos, data inválida, sem focos	TestCase	3
SensibilidadeTest	OAT 25 sim., custo, monotonicidade, reversal	SimpleTestCase	5
Total			**46**

Tabela 10: Resultados dos testes de sensibilidade OAT do IgnisGeo.

Métrica	**Resultado**	**Limiar**
Simulações OAT ($K \times L = 5 \times 5$)	25	—
Estabilidade Top 1 (IS_1)	100,0%	$> 70\%$
Estabilidade Top 3 (IS_3)	84,0%	$> 60\%$
Rank Reversal (dominada removida)	Ausente	Ausente
Custo C5 penaliza área úmida	Confirmado	—
Monotonicidade (dominante = #1)	Confirmado	—

Executado em 0,005s (SimpleTestCase, sem banco). Suite completa: 0,330s (com banco PostgreSQL/PostGIS via Docker).

```
python manage.py test queimadas.test_sensibilidade -v 2

# Com cobertura de codigo
pip install coverage
coverage run manage.py test queimadas
coverage report -m
```

11. Robustez e Decisões de Qualidade

11.1. Mapa de Decisões Arquiteturais

Tabela 11: Decisões de qualidade e justificativas técnicas.

Decisão	Alternativa descartada	Justificativa
TOPSIS Fuzzy sem NumPy	SciPy TOPSIS	Zero dependência externa no núcleo
Centroide PostGIS	Shapefile IBGE	Sem arquivo externo; geometria derivada dos dados
Canvas 2D puro	Chart.js / D3.js	Sem framework de gráficos; controle total do render
SimpleTestCase sensib.	TestCase com DB	Sem banco; executa em 5ms sem Docker
delete-then-insert	UPDATE individual	Estado parcial nunca visível pela API
ignore_conflicts=True	Verificação prévia	Reimportação segura sem query de leitura extra
Percentil de ranking	Limiar fixo de CC_i	Escala com n; garante ≥ 1 CRITICO para qualquer n
Redis alpine 40 MB	RabbitMQ 200+ MB	Menor footprint para caso de uso atual

11.2. Proteção contra Estado Parcial

Padrão delete-then-insert

O endpoint `POST /api/calcular-topsis/` executa:

1. `AreaRisco.objects.all().delete()` — remove ranking anterior
2. `AreaRisco.objects.bulk_create(novas_areas)` — persiste novo ranking

As duas operações ocorrem na mesma transação Django. A API nunca retorna um ranking em estado intermediário, pois a `RankingView` não é chamada entre os dois passos.

11.3. Robustez dos Endpoints GET

Todos os endpoints `GET` do IgnisGeo aceitam os filtros opcionais: `estado`, `bioma`, `data_inicio`, `data_fim`. A ausência de qualquer filtro retorna todos os dados sem erro — não há parâmetro obrigatório em operações de leitura.

Limites defensivos implementados:

- `FocosGeoJSONView`: máximo de 5.000 features por requisição
- `RankingView`: `max_page_size=100`
- `graficos_dados_view`: amostra de até 500 scores por nível
- `correlacao_view`: retorna HTTP 400 para $N < 3$ áreas

12. Estrutura de Diretórios do Projeto

```
ignisGeo/
    backend/
        queimadas/
            models.py           <- FocoQueimada, AreaRisco (GeoDjango)
            serializers.py      <- DRF serializers (5 criterios)
            views.py            <- 10 endpoints REST
            urls.py             <- Roteamento de endpoints
            topsis_fuzzy.py     <- Nucleo MCDM (Python puro)
            tasks.py            <- Celery task (importar_csv_inpe)
            tests.py            <- 41 testes de integracao
            test_sensibilidade.py <- 5 testes OAT (SimpleTestCase)
            migrations/         <- Migrações Django
        docs/
            queimadas/
                topsis_fuzzy.html <- Documentacao pdoc (NumPy style)
        backend/
            settings.py         <- Configuracao Django + PostGIS
            celery.py           <- Configuracao Celery
    frontend/
        src/
            components/
                MapaQueimadas.vue   <- Leaflet + camadas GeoJSON
                GraficosView.vue    <- 8 graficos Canvas 2D
                PainelRanking.vue   <- Tabela paginada
            stores/
                queimadas.js        <- Pinia store central
            api.js                  <- Axios com baseURL
        index.html
        vite.config.js
    docker-compose.yml
    docs/
        ahp_generico.drawio     <- Diagrama AHP generico
        ignisgeo_diagramas.drawio <- UML + 2 sequencias
```

13. Resumo da Arquitetura Final

Tabela 12: Indicadores finais do IgnisGeo — V1.2026.

Indicador	Valor
Focos importados	3.876.956
Unidades federativas	27 (26 estados + DF)
Biomas	6 (Cerrado, Amazônia, Caatinga, Mata Atlântica, Pantanal, Pampa)
Período	Janeiro/2025 a Abril/2026 (16 meses)
Fontes orbitais	12 satélites (MODIS 1km + VIIRS 375m + GEO)
Áreas ranqueadas	6.195 municípios
Critérios TOPSIS	5 (C1–C5; 4 benefício + 1 custo)
Testes automatizados	46 (0 falhas, 0,330s)
Endpoints REST	10
Containers Docker	5
Gráficos Canvas 2D	8 (G1–G8)
Estabilidade OAT Top1	100% (25/25 simulações)
Dependências externas	0 no núcleo analítico (Python puro)

Referências

[1] CHEN, C. T. Extensions of the TOPSIS for group decision-making under fuzzy environment. **Fuzzy Sets and Systems**, v. 114, n. 1, p. 1–9, 2000.

[2] FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures**. Tese (Doutorado) — University of California, Irvine, 2000.

[3] BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. 3. ed. Boston: Addison-Wesley, 2012.

[4] INSTITUTO NACIONAL DE PESQUISAS ESPACIAIS (INPE). **BDQueimadas — Banco de Dados de Queimadas**. São José dos Campos: INPE, 2024. Disponível em: `https://queimadas.dgi.inpe.br`. Acesso em: abr. 2026.

[5] CELERY PROJECT. **Celery: Distributed Task Queue**. Disponível em: `https://docs.celeryq.dev`. Acesso em: abr. 2026.

[6] POSTGIS DEVELOPMENT TEAM. **PostGIS 3.4 Documentation**. Disponível em: `https://postgis.net/docs`. Acesso em: abr. 2026.